雲原生 HW3 - Testing

資管三 B12705041 李倢安

April 8, 2026

Test 1：Backend —POST /api/v1/todos 成功建立 Todo

測試檔案：

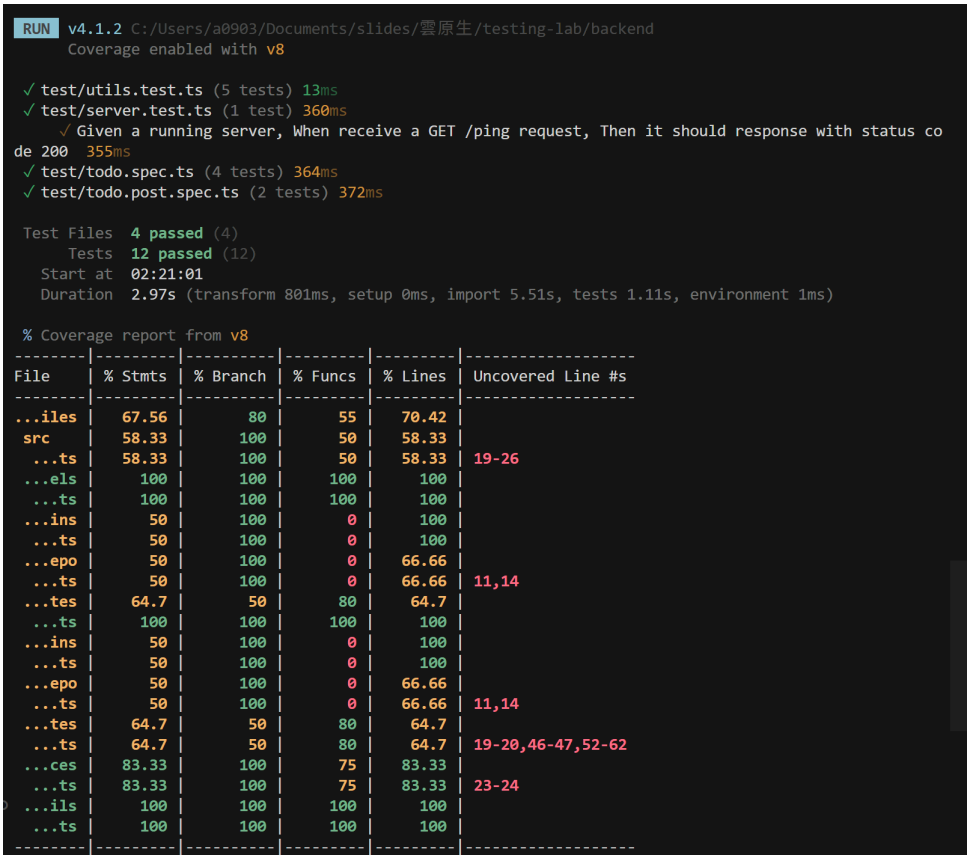
backend/test/todo.post.spec.ts

```
import { afterAll, afterEach, beforeAll, describe, expect, test, vi } from
    'vitest'
import { serverOf } from '../src/server'
import * as TodoRepo from '../src/repo/todo'
import { FastifyInstance } from 'fastify'
import { Todo } from '../src/types/todo'

describe('Todo POST API Testing', () => {
  let server: FastifyInstance

  beforeAll(async () => {
    server = serverOf()
    await server.ready()
  })

  afterAll(async () => {
    await server.close()
  })

  afterEach(() => {
    vi.resetAllMocks()
  })

  test('Given valid name and description, When receive a POST /api/v1/todos
      request, Then it should response with status code 201 and the created
      todo', async () => {
```

```
    // arrange: mock the repo function to return the created todo
    const createdTodo: Todo = {
      id: 'mock-id-1',
      name: 'Buy groceries',
      description: 'Go to the supermarket',
      status: false
    }
    const createSpy = vi.spyOn(TodoRepo, 'createTodo').mockResolvedValue(
        createdTodo)

    // act: send POST /api/v1/todos with valid body
    const response = await server.inject({
      method: 'POST',
      url: '/api/v1/todos',
      payload: JSON.stringify({ name: 'Buy groceries', description: 'Go to
          the supermarket' }),
      headers: { 'Content-Type': 'application/json' }
    })

    // assert: status 201 and body contains the created todo
    expect(response.statusCode).toBe(201)
    const result = JSON.parse(response.body)['todo']
    expect(result).toStrictEqual(createdTodo)

    // assert: repo was called exactly once with the correct payload
    expect(createSpy).toHaveBeenCalledTimes(1)
    expect(createSpy).toHaveBeenCalledWith(
      expect.objectContaining({ name: 'Buy groceries', description: 'Go to
          the supermarket' })
    )
  })

  test('Given the repo throws an error, When receive a POST /api/v1/todos
      request, Then it should response with status code 500', async () => {
    // arrange: mock the repo to throw
    vi.spyOn(TodoRepo, 'createTodo').mockRejectedValue(new Error('DB
        connection failed'))

    // act
    const response = await server.inject({
      method: 'POST',
      url: '/api/v1/todos',
      payload: JSON.stringify({ name: 'Buy groceries', description: 'Go to
```

```
          the supermarket' }),
      headers: { 'Content-Type': 'application/json' }
    })

    // assert: server should swallow the error and return 500
    expect(response.statusCode).toBe(500)
  })
})
```

測試報告截圖如下：



```
RUN  v4.1.2 C:/Users/a0903/Documents/slides/雲原生/testing-lab/backend
     Coverage enabled with v8

 ✓ test/utils.test.ts (5 tests) 13ms
 ✓ test/server.test.ts (1 test) 360ms
     ✓ Given a running server, When receive a GET /ping request, Then it should response with status co
de 200  355ms
 ✓ test/todo.spec.ts (4 tests) 364ms
 ✓ test/todo.post.spec.ts (2 tests) 372ms

 Test Files  4 passed (4)
      Tests  12 passed (12)
   Start at  02:21:01
   Duration  2.97s (transform 801ms, setup 0ms, import 5.51s, tests 1.11s, environment 1ms)

 % Coverage report from v8
--------|---------|----------|---------|---------|-------------------
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
--------|---------|----------|---------|---------|-------------------
...iles |   67.56 |       80 |      55 |   70.42 |
 src    |   58.33 |      100 |      50 |   58.33 |
  ...ts |   58.33 |      100 |      50 |   58.33 | 19-26
 ...els |     100 |      100 |     100 |     100 |
  ...ts |     100 |      100 |     100 |     100 |
 ...ins |      50 |      100 |       0 |     100 |
  ...ts |      50 |      100 |       0 |     100 |
 ...epo |      50 |      100 |       0 |   66.66 |
  ...ts |      50 |      100 |       0 |   66.66 | 11,14
 ...tes |    64.7 |       50 |      80 |    64.7 |
  ...ts |     100 |      100 |     100 |     100 |
 ...ins |      50 |      100 |       0 |     100 |
  ...ts |      50 |      100 |       0 |     100 |
 ...epo |      50 |      100 |       0 |   66.66 |
  ...ts |      50 |      100 |       0 |   66.66 | 11,14
 ...tes |    64.7 |       50 |      80 |    64.7 |
  ...ts |    64.7 |       50 |      80 |    64.7 | 19-20,46-47,52-62
 ...ces |   83.33 |      100 |      75 |   83.33 |
  ...ts |   83.33 |      100 |      75 |   83.33 | 23-24
 ...ils |     100 |      100 |     100 |     100 |
  ...ts |     100 |      100 |     100 |     100 |
--------|---------|----------|---------|---------|-------------------
```

圖 1: todo.post.spec.ts 測試報告

說明：

測試工具

層級	工具	版本	用途
Backend	Vitest	v4.1.2	單元 / 整合測試框架
Backend	`@vitest/coverage-v8`	v4.1.2	程式碼覆蓋率報告
Backend	Fastify `server.inject()`	—	不需啟動真實 HTTP 伺服器即可發送請求

測試場景（Given / When / Then）

角色	說明
Given	使用 `vi.spyOn(TodoRepo, 'createTodo').mockResolvedValue(...)` 模擬資料庫寫入，不需連接真實 MongoDB
When	透過 `server.inject()` 對 `POST /api/v1/todos` 送出包含 `name` 與 `description` 的 JSON body
Then	HTTP status 為 201，response body 的 `todo` 欄位與 mock 回傳值完全一致；且 `createTodo` 恰好被呼叫一次，且呼叫參數包含正確的欄位

- 測試主體：`TodoRouter`（路由層）→ `addTodo` service（服務層），資料層以 spy 隔離
- Mock / 隔離策略：
  - 使用 `vi.spyOn` 替換 `TodoRepo.createTodo`，完全跳過 Mongoose / MongoDB
  - `afterEach` 呼叫 `vi.resetAllMocks()`，確保各測試案例互不干擾
  - `server.inject()` 在記憶體內模擬 HTTP 請求，不需啟動網路監聽，測試速度快且無副作用
- 想驗證的東西：
  1. 路由是否正確將 request body 傳遞給 service，再由 service 傳給 repo
  2. 成功建立時回傳 HTTP 201（而不是 200）
  3. response body 結構正確（包在 ‘todo‘ key 下）
  4. `createTodo` 被呼叫的次數與參數正確（防止漏呼叫或傳錯參數）

執行指令：

```
cd backend
npm test
```

Test 2：Frontend E2E —點擊 Complete 後 Todo 切換至完成狀態

測試檔案：

frontend/tests/todo.spec.ts（第 91 行起）

```
test('Given a todo with status false, When click Complete, Then the todo title should have line-through style and Complete button should be hidden', async ({ page }) => {
    const todo = {
        id: 'mock-id-complete',
        name: 'Exercise',
        description: 'Go for a run',
        status: false,
    };
    const completedTodo = { ...todo, status: true };

    // 1. Mock initial GET: return one incomplete todo
    await page.route('**/api/v1/todos', async (route) => {
        await route.fulfill({
            status: 200,
            contentType: 'application/json',
            body: JSON.stringify({ todos: [todo] }),
        });
    });

    // 2. Mock PUT (update status) → 200
    await page.route(`**/api/v1/todos/${todo.id}`, async (route) => {
        if (route.request().method() === 'PUT') {
            await route.fulfill({
                status: 200,
                contentType: 'application/json',
                body: JSON.stringify({ todo: completedTodo }),
            });
        }
    });

    await page.goto(BASE_URL);

    // 3. Verify the todo is initially visible with no line-through
    const heading = page.getByRole('heading', { name: 'Exercise' });
    await expect(heading).toBeVisible();
    await expect(heading).not.toHaveClass('line-through');
```

```

        // 4. Re-route GET to return the completed todo before clicking Complete
        await page.route('**/api/v1/todos', async (route) => {
            await route.fulfill({
                status: 200,
                contentType: 'application/json',
                body: JSON.stringify({ todos: [completedTodo] }),
            });
        });

        // 5. Click Complete button
        await page.getByRole('button', { name: 'Complete' }).click();

        // 6. Assert: title has line-through CSS class
        await expect(heading).toHaveClass('line-through');

        // 7. Assert: Complete button is hidden (has hide-button class, which sets display:none)
        const completeButton = page.locator('button', { hasText: 'Complete' });
        await expect(completeButton).toBeHidden();
    });
```

測試報告截圖如下：

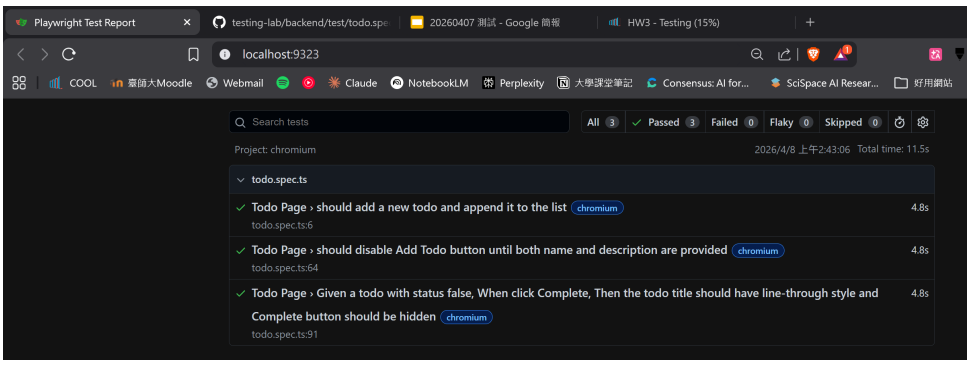


圖 2: todo.spec.ts E2E 測試報告

說明：

測試工具

層級	工具	版本	用途
Frontend	Playwright	v1.51.0	End-to-End 瀏覽器自動化測試框架
Frontend	Playwright `page.route()`	—	攔截並 mock 瀏覽器發出的 API 請求

測試場景（Given / When / Then）

角色	說明
Given	以 `page.route('**/api/v1/todos', ...)` mock 初始 GET 回傳 1 筆 `status:false` 的 Todo，頁面載入後即看到"Exercise" 項目，標題無刪除線
When	對 `PUT /api/v1/todos/:id` route 注入 200 回應；再更新 GET mock 回傳 `status:true`；最後點擊 `Complete` 按鈕觸發 `updateTodo()` → `fetchTodos()`
Then	標題 `<h1>` 套用 CSS class `line-through`（文字顯示刪除線）；`Complete` 按鈕取得 class `hide-button`（`display:none`），`toBeHidden()` 通過

- 測試主體：React 元件 `TodoItem`（`Complete` 按鈕、標題/描述的 class 邏輯）+ `App`（`handleUpdateTodo` 流程）+ API 互動（`axios PUT → GET`）
- Mock / 隔離策略：
  - `page.route()` 在瀏覽器層攔截 API 呼叫，完全不需要真實 Backend 或 MongoDB
  - 分兩階段 mock GET：第一階段回傳未完成狀態（初始渲染）、第二階段回傳完成狀態（點擊後 refetch）
  - 每個測試案例各自設置 route handler，Playwright 自動在測試結束後清除
- 想驗證的東西：
  1. 初始狀態：頁面正確渲染 `status:false` 的 Todo，無刪除線
  2. 點擊 `Complete` 後：UI 正確根據 `status:true` 套用 `line-through` class（視覺反饋）
  3. 點擊 `Complete` 後：`Complete` 按鈕被 CSS class `hide-button`（`display:none`）隱藏，防止重複點擊
  4. 驗證整個 UI 更新流程：`click → PUT API → GET refetch → React` 重新渲染的完整鏈路

執行指令：

```
# 終端機 1：啟動前端 dev server
cd frontend
npm run dev
```

```

# 終端機 2：執行 E2E 測試
cd frontend
npx playwright test --project=chromium tests/todo.spec.ts

# 產生並開啟 HTML 報告
npx playwright show-report
```

實作 1：修復測試

答: 在實作 1 中，我們發現了 utils.test.ts 中的 fabonacci testing 未實作，導致測試失敗。為了修復這個問題，我們需要實作 fabonacci function 並確保它能正確計算斐波那契數列。以下是修復後的 utils.test.ts 中 fabonacci function 程式碼片段：

```
  describe('fabonacci testing', () => {
    it('should return 1 when n is 1', () => {
      expect(fabonacci(1)).toBe(1)
    })
    it('should return 1 when n is 2', () => {
      // arrange
      const n = 2

      // act
      const actual = fabonacci(n)

      // assert
      expect(actual).toBe(1)
    })
    it('should return 2 when n is 3', () => {
      // arrange
      const n = 3

      // act
      const actual = fabonacci(n)

      // assert
      expect(actual).toBe(2)
    })
  })
```

測試報告截圖如下：

```
PS C:\Users\a0903\Documents\slides\雲原生\testing-lab\backend> npm run test
> backend@1.0.0 test
> vitest run --coverage


 RUN  v4.1.2 C:/Users/a0903/Documents/slides/雲原生/testing-lab/backend
      Coverage enabled with v8

 ✓ test/utils.test.ts (5 tests) 10ms
 ✓ test/server.test.ts (1 test) 228ms
 ✓ test/todo.spec.ts (4 tests) 242ms

 Test Files  3 passed (3)
      Tests  10 passed (10)
   Start at  00:53:38
   Duration  1.87s (transform 408ms, setup 0ms, import 2.23s, tests 480ms, environme

 % Coverage report from v8
--------------|---------|----------|---------|---------|-------------------------
File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
--------------|---------|----------|---------|---------|-------------------------
All files     |   44.59 |       60 |      35 |   46.47 |
 src          |   58.33 |      100 |      50 |   58.33 |
  server.ts   |   58.33 |      100 |      50 |   58.33 | 19-26
 src/models   |     100 |      100 |     100 |     100 |
  todo.ts     |     100 |      100 |     100 |     100 |
 src/plugins  |      50 |      100 |       0 |     100 |
  mongodb.ts  |      50 |      100 |       0 |     100 |
 src/repo     |      50 |      100 |       0 |   66.66 |
  todo.ts     |      50 |      100 |       0 |   66.66 | 11,14
 src/routes   |   26.47 |        0 |      40 |   26.47 |
  todo.ts     |   26.47 |        0 |      40 |   26.47 | 19-20,25-31,36-47,52-62
 src/services |      50 |      100 |      25 |      50 |
  todo.ts     |      50 |      100 |      25 |      50 | 12-15,19-20,23-24
 src/utils    |     100 |      100 |     100 |     100 |
  math.ts     |     100 |      100 |     100 |     100 |
--------------|---------|----------|---------|---------|-------------------------
```

圖 3: 修復後的 fabonacci testing 報告

實作 2：透過 AI 協助你完成 todo.spec.ts 裡的測試

答: 以下是修復後的 todo.spec.ts 中的程式碼片段：

```
test('Given a valid ID and status, When receive a PUT /api/v1/todos/:id
    request, Then it should response the updated todo object', async ()
    => {
// arrange: mock the repo function to return an updated todo object
const updatedTodo = {
  id: '1',
  name: 'todo 1',
  description: 'description 1',
  status: true
}
```

```
    vi.spyOn(TodoRepo, 'updateTodoById').mockImplementation(async (id, status) => {
      if (id === '1') {
        return { ...updatedTodo }
      }
      return null
    })

    // act: receive a PUT /api/v1/todos/:id request
    const response = await server.inject({
      method: 'PUT',
      url: '/api/v1/todos/1',
      payload: JSON.stringify({ status: true }),
      headers: { 'Content-Type': 'application/json' }
    })

    // assert: response should be the updated todo object
    expect(response.statusCode).toBe(200)
    const result = JSON.parse(response.body)['todo']
    expect(result).toStrictEqual(updatedTodo)
  })

  test('Given an invalid ID, When receive a PUT /api/v1/todos/:id request, Then it should response with status code 404', async () => {
    // arrange: mock the repo function to return null
    vi.spyOn(TodoRepo, 'updateTodoById').mockImplementation(async (id, status) => {
      return null
    })

    // act: receive a PUT /api/v1/todos/:id request
    const response = await server.inject({
      method: 'PUT',
      url: '/api/v1/todos/does-not-exist',
      payload: JSON.stringify({ status: false }),
      headers: { 'Content-Type': 'application/json' }
    })

    // assert: response should with status code 404
    expect(response.statusCode).toBe(404)
  })
```

測試報告截圖如下：

```
PS C:\Users\a0903\Documents\slides\雲原生\testing-lab\backend> npm run test
> backend@1.0.0 test
> vitest run --coverage


 RUN  v4.1.2 C:/Users/a0903/Documents/slides/雲原生/testing-lab/backend
      Coverage enabled with v8

 ✓ test/utils.test.ts (5 tests) 21ms
 ✓ test/server.test.ts (1 test) 442ms
     ✓ Given a running server, When receive a GET /ping request, Then it should response with status code 200  434ms
 ✓ test/todo.spec.ts (4 tests) 482ms

 Test Files  3 passed (3)
      Tests  10 passed (10)
   Start at  01:36:04
   Duration  3.72s (transform 682ms, setup 0ms, import 4.66s, tests 945ms, environment 1ms)

 % Coverage report from v8
--------------|---------|----------|---------|---------|-------------------
File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
--------------|---------|----------|---------|---------|-------------------
All files     |   56.75 |       80 |      45 |   59.15 |
 src          |   58.33 |      100 |      50 |   58.33 |
  server.ts   |   58.33 |      100 |      50 |   58.33 | 19-26
 src/models   |     100 |      100 |     100 |     100 |
  todo.ts     |     100 |      100 |     100 |     100 |
 src/plugins  |      50 |      100 |       0 |     100 |
  mongodb.ts  |      50 |      100 |       0 |     100 |
 src/repo     |      50 |      100 |       0 |   66.66 |
  todo.ts     |      50 |      100 |       0 |   66.66 | 11,14
 src/routes   |   47.05 |       50 |      60 |   47.05 |
  todo.ts     |   47.05 |       50 |      60 |   47.05 | 19-20,25-31,46-47,52-62
 src/services |   66.66 |      100 |      50 |   66.66 |
  todo.ts     |   66.66 |      100 |      50 |   66.66 | 12-15,23-24
 src/utils    |     100 |      100 |     100 |     100 |
  math.ts     |     100 |      100 |     100 |     100 |
--------------|---------|----------|---------|---------|-------------------
```

圖 4: todo.spec.ts 測試報告